

Adaptive Traffic Signal Control via Reinforcement Learning

Gaurang Ghadi¹ Tapan Shashikant Patil¹

¹Northeastern University

Problem

Traffic signal control in most cities relies on fixed-time controllers that ignore real-time traffic conditions, causing unnecessary delays. We apply deep reinforcement learning to learn adaptive signal policies at a 2-way single intersection using SUMO-RL v1.4.5, comparing multiple RL algorithms against fixed-time and planning baselines. We additionally explore a bus priority extension that differentially weights waiting time by vehicle type.

State, Actions & Rewards

- State $S \in \mathbb{R}^{21}$ — observed at each decision step:

- Current traffic signal phase
- Minimum green time elapsed indicator
- Per-lane vehicle density (8 lanes)
- Per-lane queue length (8 lanes)

- Action — discrete, $|A| = 4$:

$A \in \{\text{N-S through, N-S left, E-W through, E-W left}\}$

Min green: 5s, max green: 50s.

- Reward — defined by SUMO-RL:

$$R = -\Delta(\text{system total waiting time})$$

Experiment Setup

- Environment: SUMO v1.21.0 with SUMO-RL v1.4.5 Gym interface
- Intersection: 2-way single intersection
- Simulation horizon: 100,000 seconds per episode
- Episodes per seed: 5
- Seeds: 42, 123, 456, 789, 1000
- Evaluation metric: System total waiting time (seconds), averaged across 5 seeds

Baselines

SB3 DQN — Stable-Baselines3 library implementation of DQN. Serves two purposes: validates our from-scratch implementations and establishes a benchmark representing library-level engineering.

Parameter	Value
Learning rate	0.001
Target update	500 steps
Epsilon	0.05 $\rightarrow$ 0.01 (no decay)
Train frequency	every step
Learning starts	0
Policy network	MlpPolicy (SB3 default)
Simulation	100,000s, 5 seeds

Key difference: SB3 uses a fixed low epsilon (0.05) from the start with no exploration decay, training every step. Our implementations start at $\epsilon = 1.0$ and decay, allowing more exploration early in training.

Our Algorithms

- DQN v2** — After our initial implementation (DQN v1), we tuned hyperparameters following Mnih et al. (2015), reducing mean waiting time from 2,197s to 522s — a 4x improvement with no architectural changes.

Parameter	Value
Learning rate	0.00025
Replay buffer	10,000
Target update	every 500 steps
Train frequency	every 4 steps
Epsilon	1.0 $\rightarrow$ 0.1
Discount γ	0.99
Hidden layers	2 $\times$ 64, ReLU

Initialize Q_main, Q_target with same weights

For each step:

a = ϵ -greedy(Q_main(s))

observe r, s'

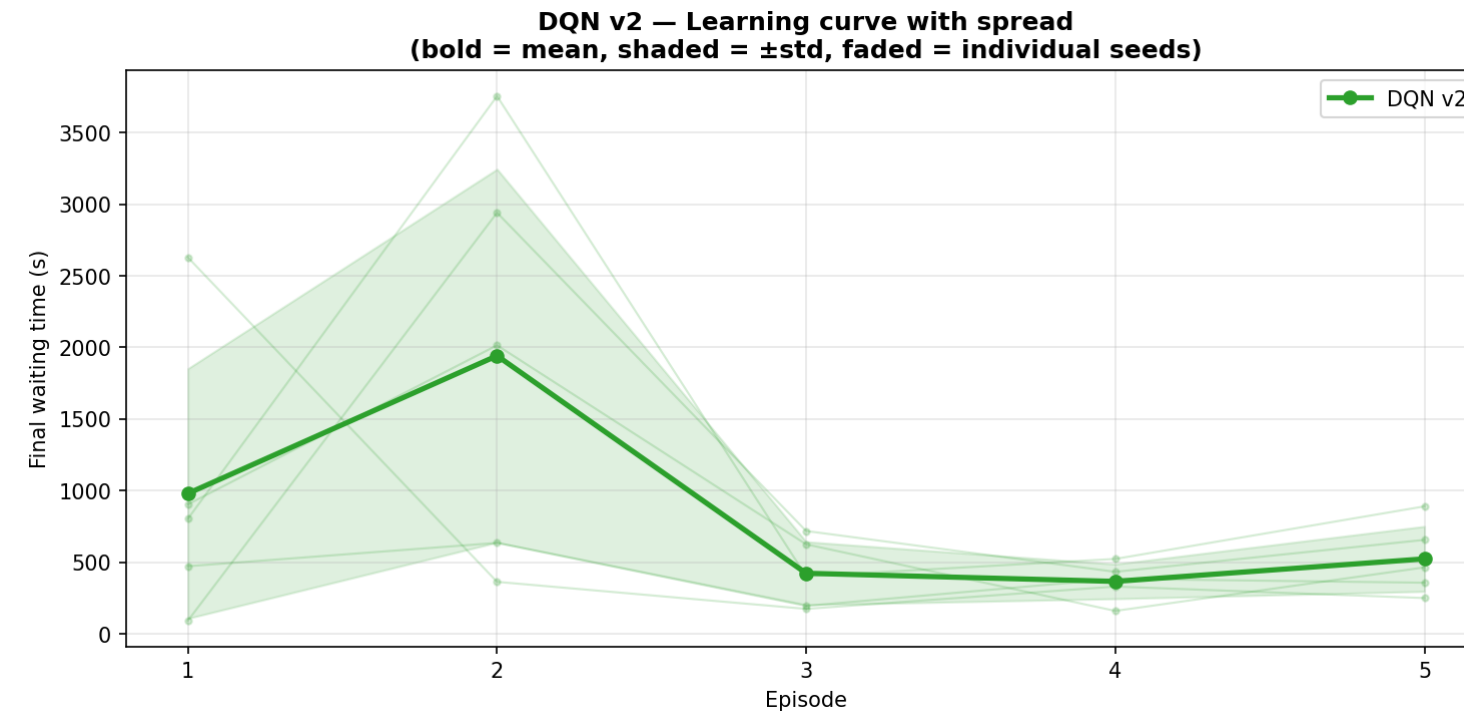
store (s, a, r, s') in replay buffer

sample minibatch from buffer

y = $r + \gamma \cdot \max_a Q_target(s', a)$

update Q_main to minimize (y - Q_main(s, a))²

every 500 steps: Q_target $\leftarrow$ Q_main



- DDQN** — Builds on DQN v2 with one key algorithmic change: decouples action selection from value estimation, reducing Q-value overestimation bias inherent in standard DQN.

Parameter	Value
Learning rate	0.00025
Replay buffer	10,000
Target update	every 500 steps
Train frequency	every 4 steps
Epsilon	1.0 $\rightarrow$ 0.1
Discount γ	0.99
Hidden layers	2 $\times$ 64, ReLU

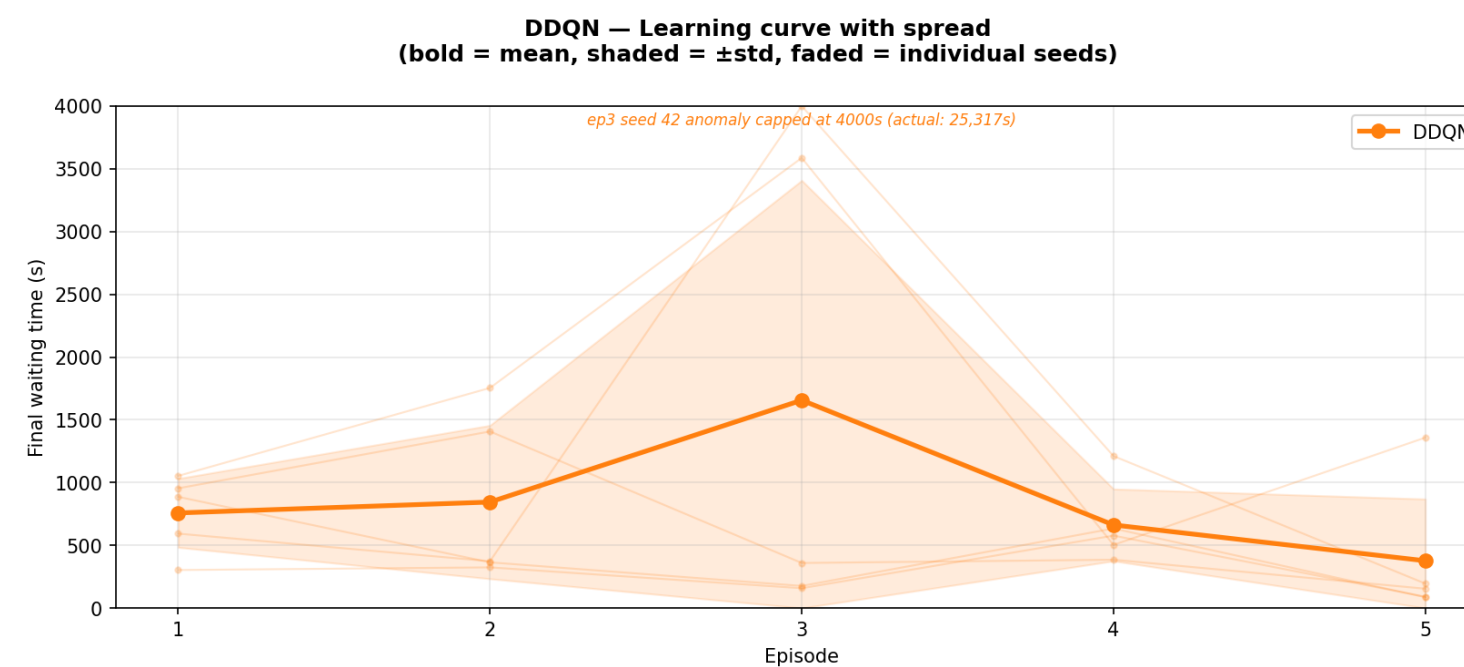
DQN target:

$$y = r + \gamma \cdot \max_a Q_{\text{target}}(s', a)$$

DDQN target:

$$a^* = \arg \max_a Q_{\text{main}}(s', a)$$

$$y = r + \gamma \cdot Q_{\text{target}}(s', a^*)$$



- Linear SARSA** — Classical RL baseline using semi-gradient SARSA. No neural network - estimates Q-values as a linear combination of binary tile features. On-policy: updates from the action actually taken, not the greedy max.

Parameter	Value
Alpha	0.05 / num_tilings
Discount γ	0.99
Epsilon	0.3 $\rightarrow$ 0.01 (decay 0.9999)
Tilings	8
Tiles per dim	4
Hash size	4096 per tiling

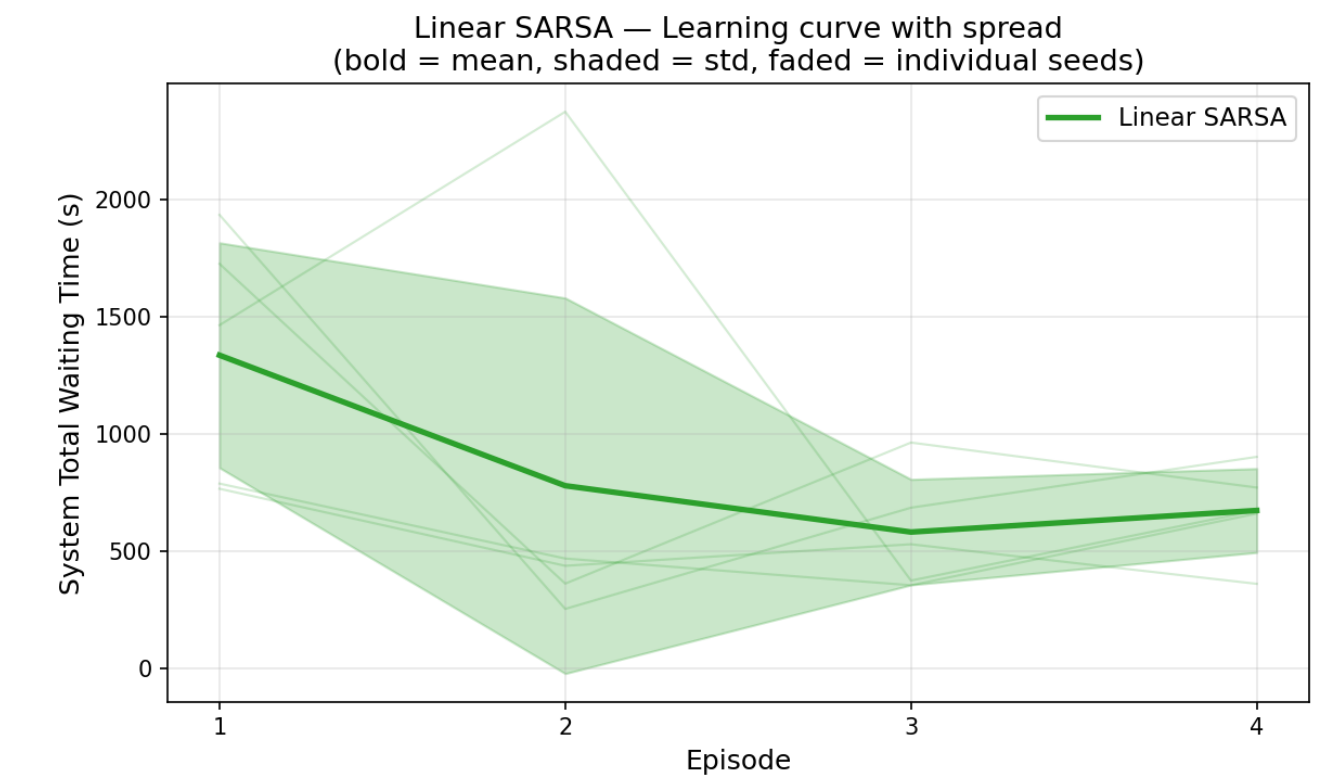
Q-value:

$$Q(s, a) = \sum_{f \in \text{active}} w[a][f]$$

SARSA update:

$$w[a][f] += \alpha \cdot (r + \gamma \cdot Q(s', a') - Q(s, a))$$

where a' is the actual next action (on-policy).



Bus Priority Extension

Gymnasium wrapper augments observation ($\mathbb{R}^{21} \rightarrow \mathbb{R}^{31}$) with per-lane bus counts, total and max bus wait. **Reward — person-delay formulation:**

$$PD(t) = W_{\text{car}} \times 1.5 + \beta \times W_{\text{bus}} \times 40 \quad R(t) = \frac{PD(t-1) - PD(t)}{100}$$

Config	Wait (s/veh)	Throughput	Std
No priority	8.66	69,187	2.66
$\beta = 0.1$	158.62	56,908	180.77
$\beta = 1.0$	99.16	54,683	110.46
$\beta = 2.0$	10.10	64,917	4.52

$\beta=2.0$: stable config - near-baseline wait, 94% throughput. Lower β destabilizes DQN (std>100). Linear FA more robust (std < 1) but plateaus at ~14s.

Results

Algorithm	Mean Wait (s)	Per Vehicle (s/veh)
DQN v1	2,197	21.24
Linear SARSA	675	12.65
DQN v2	522	7.96
DDQN	377	7.42
SB3 DQN	294	4.69

- Hyperparameter tuning produced a 4x improvement over naive DQN — larger than any algorithmic improvement
- SB3's advantage reflects library-level engineering — prioritized experience replay and gradient clipping — rather than any fundamental algorithmic difference
- Linear SARSA (675s) outperforms DQN v1 (2,197s) — linear methods beat poorly tuned deep RL

Plan to Finish

- Finalizing results across all 5 seeds for all algorithms
- Investigate whether algorithm rankings generalize across intersection complexity